

Blueprint: Real-time Collaborative Code Editor

Frontend (Editor UI)

- Framework: React (or Next.js)
- Editor Library: Monaco Editor (engine behind VS Code) or CodeMirror 6
- State Management: Redux / Zustand for local editor state
- WebSocket Client: socket.io-client or native WebSocket API

Backend (Real-time Sync)

- Node.js + Express (or NestJS for structure)
- WebSockets with Socket.IO (handles rooms, reconnections easily)
- Collaboration Algorithm:
 - - Operational Transformation (OT) → used by Google Docs
 - - CRDTs (Conflict-free Replicated Data Types) → used by Figma/Yjs/Automerger
- Start simple: use Socket.IO broadcasting, then scale with OT/CRDT

Database (Persistence)

- MongoDB or PostgreSQL for project files, sessions, history
- Redis optional for in-memory sync if scaling to many users

Optional Add-ons

- Authentication: Firebase Auth / JWT-based system
- Execution Sandbox: Docker-based runners for multiple languages
- Versioning: Git-like snapshots of code
- Chat/Voice: WebRTC for P2P communication

Workflow (How it Works)

- User opens a project in frontend (React app)
- Editor connects to Socket.IO server
- User types → change event sent to server
- Server broadcasts changes to all clients in the same room
- Each client updates local editor → real-time sync
- Server persists code in database for reload/versioning

Example Stack (Beginner-Friendly)

- Frontend: React + Monaco Editor + socket.io-client
- Backend: Node.js + Express + Socket.IO
- Database: MongoDB
- Hosting:
 - - Backend: Heroku/Render/Vercel Functions
 - - DB: MongoDB Atlas
 - - Frontend: Vercel/Netlify

Architecture Diagram (Text)

- [User A] ----\
- \
- [User B] -----> [Node.js Server w/ Socket.IO] ---> [MongoDB]
- /
- [User C] ----/

Two Development Paths

- MVP: React + Socket.IO + Monaco Editor (no DB, just real-time)
- Advanced: Add persistence (MongoDB), CRDTs/OT, authentication, execution environment